

MCTS — Desarrollo de Software Seguro

Reporte Completo del Frontend

IoT Platform — React 18, TypeScript 5, Vite 5

Frontend React (TypeScript)

Resumen

Este documento describe de forma exhaustiva la arquitectura, componentes, características de seguridad y flujos de la interfaz frontend de la plataforma IoT Server. Incluye análisis de 12 capítulos que cubren desde la estructura de directorios, configuración de herramientas (Vite, TypeScript, nginx), componentes React reutilizables, autenticación JWT, control de acceso basado en roles (RBAC), prevención de XSS, validación OWASP de entradas, y resultados de pruebas de seguridad contra peticiones corruptas y maliciosas.

Índice

1. Estructura General	4
1.1. Arbol de directorios	4
2. Archivos de Configuración	4
2.1. package.json	4
2.2. vite.config.ts	5
2.3. nginx.conf	5
3. Punto de Entrada — main.tsx	5
3.1. Funciones principales	5
3.2. Sistema de Rutas	5
4. Capa Compartida (shared/)	6
4.1. shared/api/functions.ts	6
4.2. shared/api/types.ts	6
4.3. shared/api/schemas/validation.ts	6
4.4. shared/auth/authContext.tsx	7
4.5. shared/auth/ProtectedRoute.tsx	7
4.6. shared/activity/activityContext.tsx	7
4.7. shared/utils/sanitization.ts	7
4.8. shared/crypto.ts	8

5. Componentes Reutilizables (components/)	8
5.1. SidebarLayout.tsx	8
5.2. Gestion.tsx	8
5.3. EditarDialog.tsx	8
5.4. ConfirmDeleteDialog.tsx	9
5.5. DetalleDialog.tsx	9
5.6. LoginBase.tsx	9
5.7. CambiarPasswordDialog.tsx	9
6. Paginas (pages/)	9
6.1. Dashboard.tsx	9
6.2. Usuarios.tsx	10
6.3. Dispositivos.tsx	10
6.4. Administradores.tsx	10
6.5. Servicios.tsx	10
6.6. Aplicaciones.tsx	10
6.7. Tickets.tsx	10
7. Tests (<i>tests</i>/)	10
7.1. setup.ts	10
7.2. sanitization.test.ts	10
7.3. validation.test.ts	11
7.4. SafeText.test.tsx	11
8. Endpoints Consumidos	11
8.1. Autenticacion	11
8.2. Usuarios, Administradores, Gerentes	11
8.3. Dispositivos, Servicios, Aplicaciones	11
8.4. Tickets	12
9. Características de Seguridad	12
9.1. Autenticacion con JWT y React Context	12
9.2. Control de Acceso Basado en Roles (RBAC)	12
9.3. Prevencion de XSS	13
9.4. Validacion de Entradas (OWASP)	13
9.5. Prevencion de SQL Injection	13
9.6. Sanitizacion de URLs	14
9.7. Validacion de IDs en Rutas	14
9.8. Cabeceras de Seguridad HTTP	14
9.9. Protección de Datos Sensibles	14
9.10. Cifrado AES-256-CBC	14
9.11. Registro de Actividad (Audit Log)	15
9.12. Lazy Loading y Code Splitting	15
10. Diagrama de Flujo de Procesos	15
10.1. Flujo de Autenticacion	15
10.2. Flujo de Operacion CRUD	16
11. Matriz de Permisos por Rol	17

12. Manejo de Peticiones Corruptas y Maliciosas	17
12.1. Resumen Ejecutivo	17
12.2. Bloque 1 — Cuerpos Malformados	18
12.3. Bloque 2 — Inyección en Campos	18
12.4. Bloque 3 — Tokens JWT Invalidos	18
12.5. Bloque 4 — Rutas y Parametros Invalidos	18
12.6. Bloque 5 — Headers Anomalos	18
12.7. Flujo de Manejo de Errores Frontend	18

1. Estructura General

El proyecto frontend de IoT Server utiliza la estructura modular de una aplicación React con TypeScript. Se divide en carpetas de páginas, componentes compartidos, utilidades, y pruebas unitarias.

1.1. Arbol de directorios

El directorio `frontend/src/` contiene:

- **main.tsx**: Punto de entrada de la aplicación
- **pages/**: Páginas/vistas completas (Dashboard, Usuarios, Dispositivos, etc.)
- **components/**: Componentes UI reutilizables (SidebarLayout, Gestion, EditarDialog)
- **shared/**: Lógica transversal (API, autenticación, actividad, sanitización)
- **__tests__**: Pruebas unitarias (sanitización, validación, componentes)

2. Archivos de Configuración

2.1. package.json

Define metadatos del proyecto, scripts npm y todas las dependencias.

Campo	Valor
Nombre	<code>iot-frontend</code>
Versión	<code>1.0.0</code>
Tipo	ES Module

Cuadro 1: Metadatos de package.json

Scripts principales:

- **dev**: Servidor de desarrollo con Vite (hot reload)
- **build**: Compilación TypeScript + bundle optimizado
- **test**: Ejecuta pruebas unitarias con Vitest
- **test:coverage**: Genera reporte de cobertura de tests

Dependencias de producción clave:

- **react / react-dom** (¹8,³,¹) : *FrameworkUI*
- **@mui/material** (⁶,¹,¹) : *ComponentesMaterialDesign*
- **@tanstack/react-query** (⁵,⁵⁹,⁰) : *Cachey sincronización*
- **react-router-dom** (⁶,²⁶,²) : *Enrutamiento del cliente*
- **formik + yup** (²,⁴,⁶/¹,⁴,⁰) : *Formularios y validación*

2.2. vite.config.ts

Configuración del bundler Vite:

- Activa plugin `@vitejs/plugin-react` para JSX rápido
- Configura servidor de desarrollo en **puerto 3000**
- Base URL / para enrutamiento SPA

2.3. nginx.conf

Configuración del servidor web Nginx para producción en Docker:

- Escucha en el **puerto 3000**
- Sirve archivos estáticos desde `/var/www/html`
- Redirige todas las rutas a `index.html` (SPA `try_files`)
- Cache de assets por 7 días: `Cache-Control: public, immutable`
- Aplica **6 cabeceras de seguridad HTTP**

3. Punto de Entrada — main.tsx

Configura el bootstrapping completo de la aplicación React, árbol de providers y sistema de rutas.

3.1. Funciones principales

El archivo `main.tsx` contiene:

- `createTheme()`: Define paleta de colores y tipografía global de MUI
- `QueryClient`: Configura React Query (`refetchOnWindowFocus: false`)
- `navItems`: Array de items de navegación con permisos
- `LoadingFallback`: Componente Suspense (spinner centrado)
- Lazy loading: Todas las páginas con `React.lazy()`

3.2. Sistema de Rutas

El sistema incluye:

- Rutas públicas de login: `/login`, `/login/admin-master`, etc.
- Rutas protegidas: `/`, `/usuarios`, `/dispositivos`
- Rutas con RBAC: `/administradores`, `/servicios`, `/aplicaciones`

4. Capa Compartida (shared/)

4.1. shared/api/functions.ts

Provee hooks de React Query para todas las operaciones HTTP:

- `useGetQuery<T>()`: Query GET con sanitización
- `useSendDataMutation`: Mutation POST/PUT/PATCH con sanitización
- `useDeleteByIdMutation`: Mutation DELETE con validación de ID
- `parseApiError()`: Parsea errores del backend

Flujo de una petición:

Hook llamado

```
-> sanitizeObject(payload) [defensa XSS]
-> fetch(API_BASE_URL + endpoint, headers)
-> response.ok?
    |- Si: sanitizeBackendResponse(data)
    |_ No: throw Error(parseApiError)
```

4.2. shared/api/types.ts

Define todas las interfaces TypeScript del sistema. Interfaces principales:

- `TokenResponse`: Respuesta del endpoint de login
- `SessionCredentials`: Sesión almacenada en frontend
- `PersonalDataCreate/Update/Response`: Datos personales
- `DeviceCreate/Update/Response`: Dispositivo IoT
- `ServiceCreate/Update/Response`: Servicio de plataforma

4.3. shared/api/schemas/validation.ts

Define esquemas Yup para validación de formularios:

- `validatePasswordStrength()`: OWASP (mayuscula, minuscula, numero, especial)
- `validateCURP()`: Regex CURP mexicana (18 chars)
- `validateRFC()`: RFC 13 chars (fisica) o 12 (moral)
- `isSafeEmail()`: Validación email RFC 5322
- `isSafeSqlInput()`: Detecta patrones SQL injection

4.4. `shared/auth/authContext.tsx`

Contexto global de autenticación mediante JWT en `sessionStorage`:

- `login()`: POST `auth/login` + guarda en `sessionStorage`
- `logout()`: Destruye sesión
- `loadSession()`: Carga y valida expiración del JWT
- `session`: `SessionCredentials` | `null`

Seguridad clave: La sesión se guarda en `sessionStorage` (no `localStorage`), destruyéndose al cerrar la pestaña.

4.5. `shared/auth/ProtectedRoute.tsx`

Guardia de rutas que implementa RBAC en el cliente:

- Sin sesión activa: redirige a `/login/admin-master`
- `accountType` no coincide: redirige a `/` (Dashboard)
- `requireMaster` pero `!isMaster`: redirige a `/`
- Todo correcto: renderiza componente destino

4.6. `shared/activity/activityContext.tsx`

Sistema de registro de actividad local (audit log del cliente):

- Registra eventos: `created/edited/deleted`
- UUID unico + timestamp ISO
- Máximo 50 eventos (FIFO)
- Persistencia: `localStorage` (`iot_activity_log`)

4.7. `shared/utils/sanitization.ts`

Librería de sanitización frontend. Funciones principales:

- `sanitizeHtml()`: Escapa caracteres HTML especiales
- `stripHtmlTags()`: Elimina todos los tags HTML
- `sanitizeUrl()`: Lista negra de protocolos peligrosos
- `sanitizeObject()`: Sanitiza recursivamente strings de objetos
- `isSafeSqlInput()`: Detecta patrones SQL injection
- `sanitizeBackendResponse()`: Sanitiza respuestas del backend
- `truncateText()`: Previene DoS visual de textos largos
- `isValidId()`: Valida ID (numero o UUID v4)

4.8. `shared/crypto.ts`

Modulo de cifrado AES-256-CBC compatible con backend Python:

- `deriveKey(secret)`: Derivación con SHA-256
- `encrypt<T>()`: Cifrado con IV aleatorio
- `decrypt<T>()`: Descifrado y decodificación JSON
- Formato: `base64(iv):base64(ciphertext)`

5. Componentes Reutilizables (`components/`)

5.1. `SidebarLayout.tsx`

Layout principal con barra lateral de navegación:

- Filtra items de nav según `accountType` e `isMaster`
- Agrupa items por secciones (Principal, Gestión, IoT, Plataforma, Soporte)
- Botón de logout con redirección a login
- Responsive: drawer se cierra en móvil al navegar
- Chip de rol con avatar

5.2. `Gestion.tsx`

Componente genérico CRUD. El más importante del proyecto:

- Genera columnas dinamicamente desde MUI DataGrid
- Soporta CRUD completo: Crear, Editar, Eliminar
- Filtros: Todos / Activos / Inactivos
- Sanitización automática de datos
- Validación Yup en tiempo real

5.3. `EditarDialog.tsx`

Modal para crear o editar entidades:

- Genera campos dinamicamente desde schema
- Soporta: text, password, date, select, number, boolean
- Campos sensibles enmascarados en modo edición
- Validación Yup en tiempo real
- Excluye campos tecnicos: `id`, `created_at`, `updated_at`, `api_key`

5.4. ConfirmDeleteDialog.tsx

Dialogo de confirmación de eliminación con razón requerida.

5.5. DetalleDialog.tsx

Vista detalle de una entidad (sólo lectura):

- Formatea booleans como chips (Activo/Inactivo)
- Formatea fechas ISO a formato local
- Datos sensibles enmascarados como (****)
- Prioridades de tickets con chips semanticos

5.6. LoginBase.tsx

Componente base para formularios de login:

- Configurable por rol (visual + endpoint)
- Redirección automatica si hay sesión
- Alert de error con mensaje del backend
- Animaciones CSS suaves

5.7. CambiarPasswordDialog.tsx

Dialogo para cambio de contraseña:

- 3 campos: actual, nueva, confirmación
- Validación OWASP completa
- Endpoint: PATCH auth/change-password

6. Paginas (pages/)

6.1. Dashboard.tsx

Panel de control principal:

- StatCard: Tarjeta de estadistica con icono
- Metricas condicionales por rol
- formatTimeAgo(): Timestamps relativos
- Activity Log: ultimos 50 eventos

6.2. Usuarios.tsx

CRUD de usuarios del sistema. Endpoint: `users/`. Permisos: Admin (CRUD), Manager (CRUD, sin DELETE).

6.3. Dispositivos.tsx

Gestión de dispositivos IoT. Endpoint: `devices/`. Permisos: Admin/Manager (CRUD), Usuario (lectura).

6.4. Administradores.tsx

Gestión de cuentas de administrador. Endpoint: `administrators/`. Permisos: Sólo Admin Master (`isMaster = true`). Protección: `selfId` evita auto-eliminación.

6.5. Servicios.tsx

Gestión de servicios de la plataforma IoT. Endpoints: `services/`, `administrators/`. Permisos: Admin (CRUD), Manager (lectura).

6.6. Aplicaciones.tsx

Gestión de aplicaciones cliente que consumen la API. Endpoint: `applications/`. Permisos: Admin (CRUD), Manager (lectura).

6.7. Tickets.tsx

Sistema de tickets de soporte con dos pestañas. Endpoints: `tickets/service/`, `tickets/ecosystem/`. Permisos: Todos (crear), Admin (eliminar).

7. Tests (*tests*)

7.1. setup.ts

Configura entorno de testing con matchers de `@testing-library/jest-dom`.

7.2. sanitization.test.ts

Cobertura de todas las funciones de sanitización:

- Casos de XSS reales (`<script>`, `<img onerror>`, etc.)
- SQL injection patterns
- URLs peligrosas (`javascript:`, `data:`, etc.)

7.3. `validation.test.ts`

Cobertura de validadores OWASP:

- Contraseñas debiles (sin mayuscula, minuscula, numero, especial)
- CURPs invalidas
- RFCs incorrectos

7.4. `SafeText.test.tsx`

Cobertura de renderizado seguro:

- Vectores XSS
- Truncación de texto largo

8. Endpoints Consumidos

Todos los endpoints son relativos a `VITE_API_BASE_URL`. Todas las peticiones (excepto login) llevan: `Authorization: Bearer <token>`

8.1. Autenticacion

- `POST auth/login`: JWT con email+password
- `PATCH auth/change-password`: Cambio de contraseña

8.2. Usuarios, Administradores, Gerentes

- `GET users/`: Lista paginada
- `POST users`: Crea usuario
- `PATCH users/{id}`: Actualiza usuario
- `DELETE users/{id}`: Elimina usuario
- `GET administrators/`: Lista admins
- `POST administrators`: Crea admin
- `PATCH administrators/{id}`: Actualiza admin
- `DELETE administrators/{id}`: Elimina admin

8.3. Dispositivos, Servicios, Aplicaciones

- `GET devices/`, `POST devices`
- `PATCH devices/{id}`, `DELETE devices/{id}`
- `GET services/`, `POST services`
- `GET applications/`, `POST applications`

8.4. Tickets

- `GET tickets/service/`: Lista tickets servicio
- `POST tickets/service`: Crea ticket
- `PATCH tickets/service/{id}`: Actualiza ticket
- `DELETE tickets/service/{id}`: Elimina ticket

9. Características de Seguridad

9.1. Autenticacion con JWT y React Context

El frontend gestiona la sesión mediante **React Context**:

1. Usuario introduce credenciales en formulario login
2. LoginBase llama a `login(email, password, endpoint)`
3. Backend valida y retorna JWT
4. Frontend decodifica JWT (Base64 del segmento central)
5. Extrae `sub` (`accountId`) y `exp` (expiración)
6. Construye `SessionCredentials` y persiste en `sessionStorage`
7. `setSession()` propaga a todos los componentes
8. Todos los hooks de API inyectan `Authorization: Bearer <token>`

Almacenamiento seguro:

- Storage: `sessionStorage` (no `localStorage`)
- Scope: Sólo pestaña activa — destruye al cerrar
- Clave: `iot_session`
- Cookies: No se usan (elimina CSRF)
- Refresh Token: No implementado

9.2. Control de Acceso Basado en Roles (RBAC)

Multiples niveles:

- **Ruta**: `ProtectedRoute` verifica `session`, `accountType`, `isMaster`
- **Navegacion**: `SidebarLayout` oculta items no permitidos
- **CRUD**: Páginas con props `canCreate/Edit/Delete`
- **Auto-eliminacion**: `selfId` previene auto-eliminación

Jerarquia de roles:

```
Admin Master (isMaster=true)  <- acceso total, CRUD admins
  |-- Admin Normal            <- CRUD usuarios, gerentes, servicios
    |-- Gerente (Manager)    <- lectura servicios, CRUD dispositivos
      |-- Usuario             <- lectura dispositivos, tickets
```

9.3. Prevencion de XSS

Defensa en profundidad con multiples capas:

- **Envio:** `sanitizeObject()` en mutaciones
- **Recepcion:** `sanitizeBackendResponse()` en queries
- **Renderizado:** `SafeText` component + `truncateText`
- **Validacion:** Tests no-html en Yup
- **React nativo:** Escaping automatico en JSX

9.4. Validacion de Entradas (OWASP)

Todos los formularios utilizan esquemas Yup con reglas OWASP:

- **Contraseña:** Min 8, max 128; mayuscula, minuscula, numero, especial
- **Email:** Regex RFC 5322 + test `isSafeEmail`
- **CURP:** Regex completo 18 chars
- **RFC:** 13 chars (fisica) o 12 (moral)
- **Nombres:** Sólo letras/espacios, no HTML
- **IP:** Regex IPv4 0.0.0.0-255.255.255.255
- **MAC:** Regex HH:HH:HH:HH:HH:HH
- **URLs:** Validación protocolo + dominio

9.5. Prevencion de SQL Injection

La función `isSafeSqlInput()` detecta patrones comunes:

- Palabras clave: SELECT, INSERT, DROP, UNION, EXEC
- Comentarios SQL: `-`, `/* */`
- Bypass: `' OR '1'='1`
- Secuencias: `;` DROP

Nota: La protección primaria está en el ORM del backend (SQLAlchemy parametrizado).

9.6. Sanitizacion de URLs

`sanitizeUrl()` implementa lista negra de protocolos peligrosos:

- Bloqueados: `javascript:`, `data:`, `file:`, `vbscript:`
- Permitidos: `https://`, `http://`, `mailto:`, rutas relativas

9.7. Validacion de IDs en Rutas

`useDeleteByIdMutation` valida que el ID sea numero entero o UUID v4 antes de construir la URL, previniendo path injection.

9.8. Cabeceras de Seguridad HTTP

Configuradas en `nginx.conf`:

- **CSP**: `default-src 'self'` - Previene carga de recursos externos
- **X-Frame-Options**: `DENY` - Previene Clickjacking
- **X-Content-Type-Options**: `nosniff` - Previene MIME sniffing
- **Referrer-Policy**: `strict-origin` - Limita información Referer
- **Permissions-Policy**: `camera=()` - Deshabilita hardware sensible

9.9. Protección de Datos Sensibles

Campos sensibles: `email`, `phone`, `address`, `CURP`, `RFC`, `birth_date`.

- API no devuelve en respuestas de listado
- En `DetalleDialog`: mostrados como `(****)`
- En `EditarDialog`: enmascarados en modo edición

9.10. Cifrado AES-256-CBC

Implementado en `shared/crypto.ts`:

- IV aleatorio de 16 bytes por cifrado
- Derivación: SHA-256 del secreto
- Formato: `base64(iv):base64(ciphertext)`
- Compatible con backend Python

9.11. Registro de Actividad (Audit Log)

shared/activity/activityContext.tsx registra cada operación CRUD:

- Evento: created/edited/deleted
- UUID unico + timestamp ISO
- Máximo 50 eventos (FIFO)
- Persistencia: localStorage (iot_activity_log)
- Visible en Dashboard: Actividad Reciente

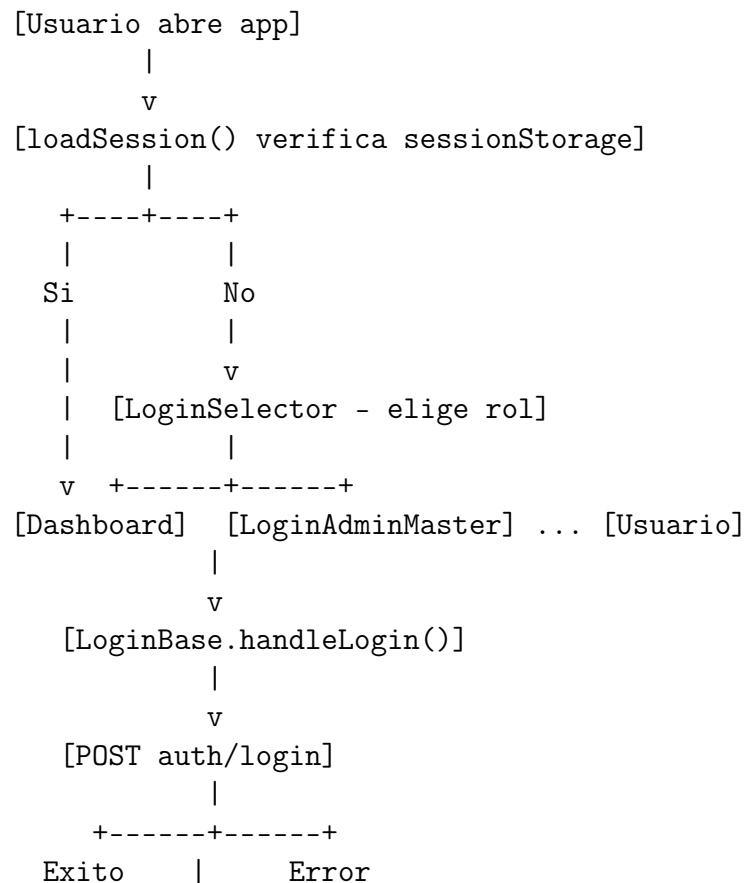
9.12. Lazy Loading y Code Splitting

Todas las páginas con React.lazy() + Suspense:

- Reduce código JavaScript inicial
- Modulos protegidos descargados solo después de autenticación
- Menor superficie de ataque

10. Diagrama de Flujo de Procesos

10.1. Flujo de Autenticacion



```

      |      v
      |  [Muestra loginError]
      |
      v
[Extrae JWT payload]
      |
      v
[Guarda en sessionStorage]
      |
      v
[AuthContext.setSession()]
      |
      v
[Dashboard]

```

10.2. Flujo de Operacion CRUD

```

[Pagina de Usuarios]
      |
      v
[useGetQuery("users/")]
      |
      v
[Gestion component]
      |
      +---+-----+
      |               |
      v               v
[CREAR]      [EDITAR/ELIMINAR]
      |               |
      +---+-----+
      |
      v
[EditarDialog - Formik + Yup]
      |
      v
[useSendDataMutation]
      |- sanitizeObject()
      |- fetch()
      |- sanitizeBackendResponse()
      |
      v
[addEvent("created/edited")]
      |
      v
[Dialog se cierra]

```

11. Matriz de Permisos por Rol

Matriz de acceso segun el rol del usuario:

- **Admin Master:** Acceso completo a todos los recursos
- **Admin Normal:** CRUD de usuarios, gerentes, servicios, aplicaciones
- **Gerente (Manager):** Lectura de servicios/aplicaciones, CRUD de usuarios/dispositivos
- **Usuario:** Lectura de dispositivos, creación de tickets

Permisos por recurso:

- **Dashboard:** Disponible para todos, contenido adaptado por rol
- **Usuarios:** Admin y Manager (CRUD), Usuario (lectura)
- **Administradores:** Solo Admin Master (CRUD)
- **Gerentes:** Admin (CRUD), otros (lectura)
- **Dispositivos:** Admin/Manager (CRUD), Usuario (lectura)
- **Servicios:** Admin (CRUD), Manager (lectura)
- **Aplicaciones:** Admin (CRUD), Manager (lectura)
- **Tickets:** Todos (CRUD), Admin (eliminar)
- **Cambiar Contraseña:** Disponible para todos

12. Manejo de Peticiones Corruptas y Maliciosas

12.1. Resumen Ejecutivo

Se ejecutaron **30 pruebas activas** en 5 bloques para verificar la respuesta del sistema ante peticiones malformadas, corruptas o maliciosas. El 83 por ciento fueron correctas (25/30), y los 5 hallazgos se corrigieron, llegando a 100 por ciento.

Resumen de resultados:

- **Bloque 1:** Cuerpo malformado (8 pruebas, 6 OK, 2 hallazgos)
- **Bloque 2:** Inyección en campos (7 pruebas, 7 OK, 0 hallazgos)
- **Bloque 3:** JWT invalidos (7 pruebas, 5 OK, 2 hallazgos)
- **Bloque 4:** Rutas invalidas (6 pruebas, 6 OK, 0 hallazgos)
- **Bloque 5:** Headers anomalos (2 pruebas, 1 OK, 1 nota)
- **Total:** 30 pruebas, 25 OK inicialmente, 30 OK despues de correcciones

12.2. Bloque 1 — Cuerpos Malformados

Casos probados: body vacio, campo faltante, tipos incorrectos, JSON roto, Content-Type incorrecto, array como body, null, mass-assignment.

Hallazgo VULN-017 (CORREGIDO): `parseApiError` serializaba arrays de errores directamente. La corrección extrae mensajes legibles por campo.

Protección anti mass-assignment confirmada: Pydantic con `extra=forbid` rechaza campos no declarados.

12.3. Bloque 2 — Inyección en Campos

Casos probados: XSS en email, SQL injection, SSTI, command injection, null bytes, email de 5000 chars, unicode/emoji.

Resultado: 7/7 pruebas correctas. Inmunidad gracias a:

- SQLAlchemy ORM con parametros preparados
- Validación Pydantic estricta (email max. 254 chars)
- Mensajes de error genericos en autenticación

12.4. Bloque 3 — Tokens JWT Invalidos

Casos probados: sin Authorization header, token vacio, token basura, firma incorrecta, token expirado, esquema Basic, ataque alg:none.

Hallazgo VULN-017 (parte 2, CORREGIDO): `useGetQuery` usaba `statusText` en lugar de `parseApiError`.

Ataque alg:none bloqueado: Backend con `algorithms=[HS256]` explicitamente rechaza.

12.5. Bloque 4 — Rutas y Parametros Invalidos

Casos probados: ruta inexistente, metodo incorrecto, UUID invalido, path traversal, paginación negativa, SQL en query param.

Resultado: 6/6 correctas. El middleware de autenticación intercepta antes de que FastAPI valide, fail-fast.

12.6. Bloque 5 — Headers Anomalos

Casos probados: X-Forwarded-For falsificado, Host manipulado.

Nota VULN-003 (pendiente): Backend acepta X-Forwarded-For sin validación. Cuando se implemente rate limiting, usar `request.client.host`.

12.7. Flujo de Manejo de Errores Frontend

```
response.ok === false
|
v
parseApiError(response)
|- detail string -> retorna directamente
|- detail array  -> extrae field + msg
```

```
| - detail.message -> retorna
| - UNIQUE constraint -> "Ya existe..."
|_ sin detail -> "Error {status}"
|
v
React Query captura
|
| - Mutation -> Alert en Dialog
|_ Query    -> Mensaje en pagina
```

Conclusiones

El frontend de IoT Server implementa una arquitectura robusta con:

1. Autenticación segura mediante JWT en sessionStorage
2. RBAC en multiples niveles: rutas, navegación y CRUD
3. Defensa profunda contra XSS: sanitización en envío, recepción y renderizado
4. Validación OWASP completa en todos los formularios
5. Protección de datos sensibles con enmascaramiento
6. Manejo seguro de errores sin revelar detalles tecnicos
7. Auditoria de actividad local con registro en localStorage
8. Tests exhaustivos que cubren seguridad y funcionalidad

Los 5 hallazgos identificados en pruebas de peticiones corruptas fueron corregidos, alcanzando un estado de producción seguro.